COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	09/10/26	**DATE SUBMITTED:**	09/14/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort.

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 8-1 in section 6.
- Create a header file for the implementation of the different sorting algorithms.
- Import this header file into your main.cpp.

A.1. Shell Sort

General Algorithm
The general algorithm for shell sort is given below.

```
shellSort(array, size)
  for interval i <- size/2n down to 1
    for each interval "i" in array
        sort all the elements at interval "i"
end shellSort
```

A.2. Merge Sort

General Algorithm
The general pseudo-code for the merge sort technique is given below.

```
Declare an array Arr of length N
If N=1, Arr is already sorted
If N>1,
Left = 0, right = N-1
Find middle = (left + right)/2
Call merge_sort(Arr,left,middle) =>sort first half recursively
Call merge_sort(Arr,middle+1,right) => sort second half recursively
Call merge(Arr, left, middle, right) to merge sorted arrays in above steps.
Exit
```

A.3. Quick Sort

```
quicksort(A, low, high)
begin
Declare array A[N] to be sorted
    low = 1st element; high = last element; pivot
if(low < high)
    begin
    pivot = partition (A,low,high);
    quicksort(A,low,pivot-1)
    quicksort(A,pivot+1,high)
    End
end
```

Code + Console Screenshot	main.cpp > main()
	1 #include <iostream>
	5
	6 // DECLARATIONS
	7 const int max = 100;
	8
	9 // FUNCTION PROTOTYPE/S
	10 void display(int arr[]);
	11 void arrayMaker(int arr[], int seed);
	12
	13 int main(){
	14 int array1[max], array2[max], array3[max];
	15 arrayMaker(array1, 67);
	16 arrayMaker(array2, 67);
	17 arrayMaker(array3, 67);
	
	62 // FUNCTION DEFINITION/S
	63 void display(int arr[]){
	64 for (int i = 0; i < max; i++) {
	65 std::cout << arr[i] << " ";
	66 }
	67 }
	68
	69 void arrayMaker(int arr[], int seed){
	70 srand(seed);
	71 for (int i = 0; i < max; i++) {
	72 arr[i] = rand() % 100 + 1;
	73 }
	74 }
	
	Terminal Output:
	Unsorted Array:
	58 67 22 48 4 45 92 67 16 8 12 53 88 42 85 94 11 92 52 70 52 39 42 65
	18 66 63 68 95 22 2 28 17 64 75 6 76 55 39 55 37 58 35 29 18 12 84 5 7 32 5 94
	5 8 85 86 49 36 23 63 47 16 91 97 60 50 63 11 29 33 34 55 89 87 91 45 8 51 69 98 20 26 78 13 20 11
	18 90 40 38 13 3 28 3 71 100 36 2 78 85

Observations	In the code above, I had decided on generating the array of values for the testing of the sorting algorithms via the use of a seed maker in a function. I also used a for loop to display the array in another function.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

```cpp
// ========================================================================

/* FOLLOWING LAB PSEUDOCODE:
shellSort(array, size)
    for interval i <= size/2n down to 1
        for each interval "i" in array
            sort all the elements at interval "i"
end shellsort
*/


template <typename T>
void shellSort(T arr[], int size){
    int interval = size / 2;

    while (interval >= 1) {
        for (int i = interval; i < size; i++) {
            T temp = arr[i];
            int j = i;

            while (j >= interval && arr[j - interval] > temp) {
                arr[j] = arr[j - interval];
                j = j - interval;
            }

            arr[j] = temp;
        }

        interval = interval / 2;
    }
}
```

Terminal Output:

```
SHELL SORT:
Unsorted Array:
58 67 22 48 4 45 92 67 16 8 12 53 88 42 85 94 11 92 52 70 52 39 42 65 18 66 63 68 95 22 2 28 17
64 75 6 76 55 39 55 37 58 35 29 18 12 84 5 7 32 5 94 5 8 85 86 49 36 23 63 47 16 91 97 60 50 63
11 29 33 34 55 89 87 91 45 8 51 69 98 20 26 78 13 20 11 18 90 40 38 13 3 28 3 71 100 36 2 78 85
Sorted Array:
2 2 3 3 4 5 5 5 6 7 8 8 8 11 11 11 12 12 13 13 16 16 17 18 18 18 20 20 22 22 23 26 28 28 29 29 3
2 33 34 35 36 36 37 38 39 39 40 42 42 45 45 47 48 49 50 51 52 52 53 55 55 55 58 58 60 63 63 63 6
4 65 66 67 67 68 69 70 71 75 76 78 78 84 85 85 85 86 87 88 89 90 91 91 92 92 94 94 95 97 98 100
```

Observations	The code above shows that of the shell sort algorithm that I had written where it follows the arranging of elements within an unsorted array by specific gaps that are according to the size of the array divided by two, over and over, until we reach a gap size of 1 where its most simplest way of sorting and what shell sort was inspired by takes place, insertion sort. The time complexity for this sorting method would be **O(n log n)** for its best case and **O(n^2)** for its worst as it makes use of nested while loops to achieve what it wants to do.

Table 8-2. Shell Sort Technique

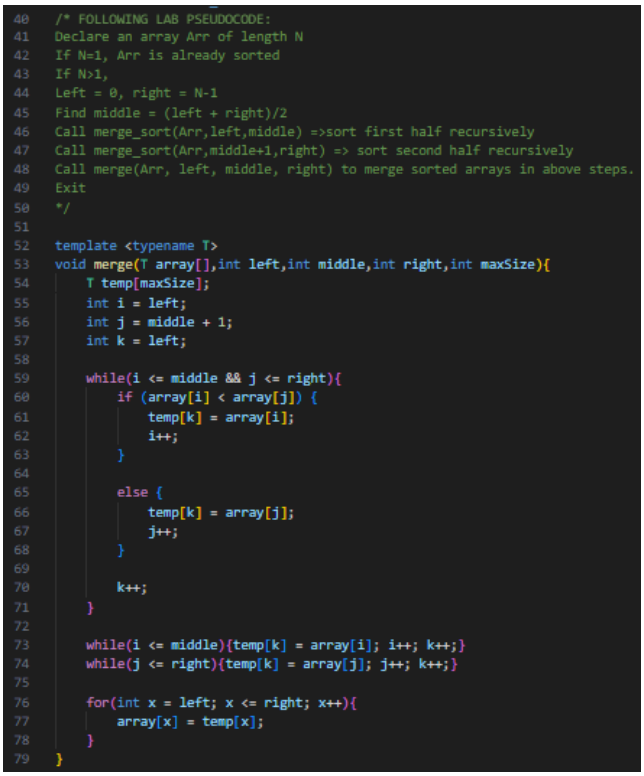
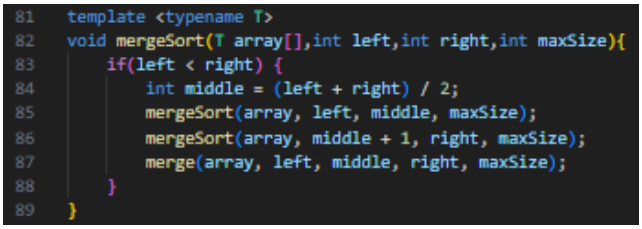
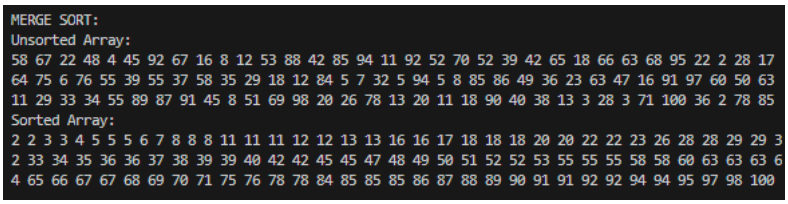
Code + Console Screenshot	  **Terminal Output:** 
Observations	The code above shows my algorithm for merge sort that I had built from the lab pseudocode. It follows the principle of 'divide and conquer' where it splits up the values in the array until they are in singular forms before merging accordingly over and over until we have the final merged sorted array in the end. The time complexity of merge sort would be **O(n log n)** for all cases as the algorithm splits the array in half recursively **O(log n)** and we merge them back together accordingly **O(n)**, hence **O(n log n)**.

Table 8-3. Merge Sort Algorithm

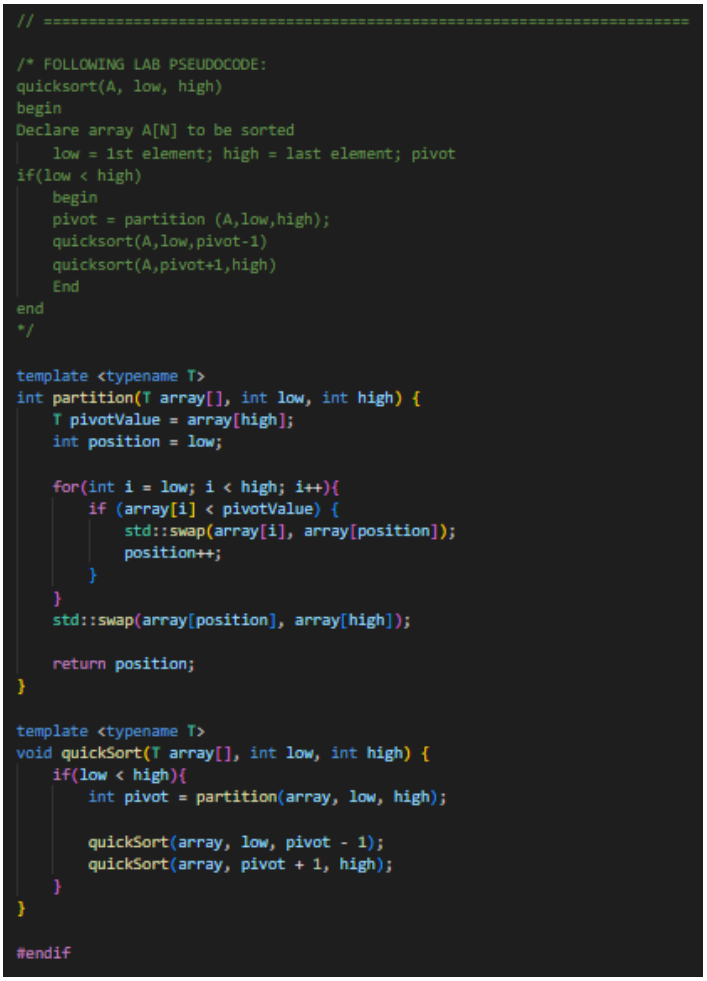
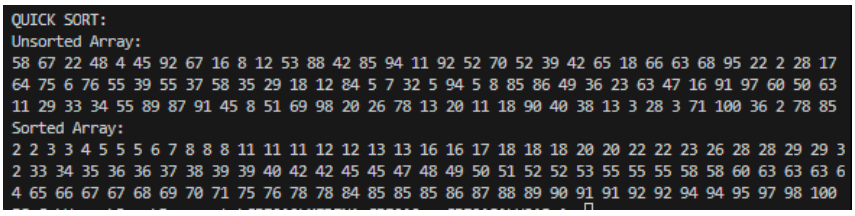
Code + Console Screenshot	 **Terminal Output:** 
Observations	This quick sort algorithm makes use of using pivots where said pivot is centered and any number higher than the pivot is put to its right sub list where the smaller values are put to the left sub list. Its time complexity is **O(n log n)** for its best case and **O(n^2)** for its worst case as the balanced partitions give **n log n** and the repeated uneven partitions give the **n^2**.

Table 8-4. Quick Sort Algorithm

2. SUPPLEMENTARY OUTPUT

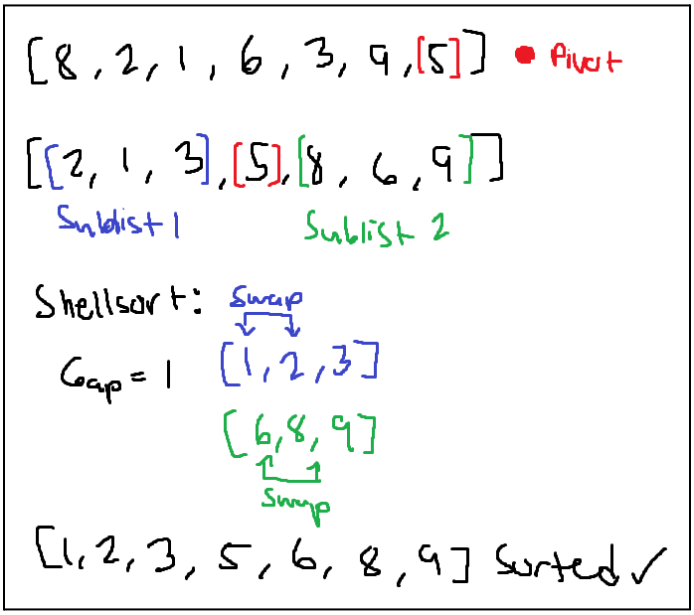
ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Problem 1 Answer:

Yes. We can definitely sort the left and right sub lists from the partition method in quick sort as we know that anything to the left of the pivot is smaller than itself and anything to the right is larger. Hence, we know that anything inside the left sublist just needs to be sorted by itself where the same goes for the right. An example would be doing shell sort on the sublists right after the pivot from quick sort.



Problem 2 Answer:

The sorting algorithm that would probably give the fastest time performance is shell sort as it does exceptionally well with small datasets, given that the array only has 15 elements and doesn't need to go over what merge sort does (which is probably the slowest) where it's recursive. Additionally, quick sort might work just as well.

Merge and quick sort have **O(n log n)** for their time complexity because merge sort divides the array into halves that are about **O(log n)** with merging taking **O(n)**. Quick sort has the partitions balanced reasonably so it also takes about **O(n log n)**.

Conclusion:

In conclusion, this activity has taught me more about sorting algorithms, especially with how we handled bubble sort, insertion sort, and selection sort the week before. I've learned based on the procedures that shell sort is kind of like the improved version of insertion sort where it takes into account the arranging of elements with the use of gaps where when it finally reaches a gap of 1, it does a simple insertion sort for the final pass. Furthermore, merge sort divides the dataset and conquers them by sorting before merging and repeating that cycle until it merges the final, fully sorted elements in the end. Finally, quick sort picks the last element as a pivot point where it creates sublists that arrange any number smaller than the pivot to the left side and any number higher to the right as the pivot is put in the center. It then does the same sort with the newly created sublists until it is fully sorted. The supplementary activities are more so conceptual questions rather than another coding session, and it goes into how other sorts are possible to apply within a valid sort and how the time complexities exactly work with the algorithms. Overall, I think I did well in the activity and my areas for improvement lie in getting the hang of how the algorithms are logically structured.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.